LAB CYCLE 1

Date:22/01/2026
Experiment No.01

Aim:Familiarization of DDL Commands

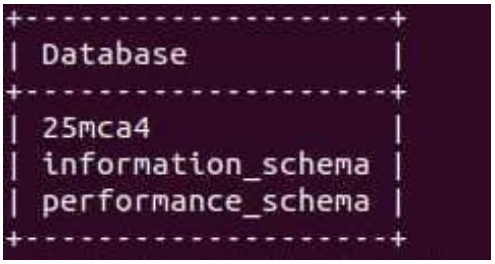
Data Definition Language (DDL) - These SQL commands are used for creating, modifying, and dropping the structure of database objects. The commands are CREATE, ALTER, DROP, RENAME, and TRUNCATE.

A. Consider the database for a college. Write SQL commands to implement the following:

1. Create a database

>>create database 25mca4;

OUTPUT:



2. Select the current database

>>USE 25mca4;

OUTPUT:

3. Create the following tables:

a) Student (roll_no integer, name varchar, dob date, address text,phone_no varchar, blood_grp varchar)

>> CREATE TABLE student(roll_no INT PRIMARY KEY, name VARCHAR(50), dob DATE, address TEXT,phone_no VARCHAR(50),blood_grp VARCHAR(10));

b) Course (Course_id integer, Course_name varchar, course_duration integer)

>> CREATE TABLE course(course_id INT PRIMARY KEY, course_name VARCHAR(50), course_duration INT);

OUTPUT:

```
mysql> show tables;
+------------------+
| Tables_in_25mca4 |
+------------------+
| course           |
| student          |
+------------------+
```

4. List all tables in the current database.

>> show tables;

OUTPUT:

```
mysql> show tables;
+------------------+
| Tables_in_25mca4 |
+------------------+
| course           |
| student          |
+------------------+
```

5. Display the structure of the Student table.

>> desc Student;

OUTPUT:

```
+-----------+-------------+------+-----+---------+-------+
| Field     | Type        | Null | Key | Default | Extra |
+-----------+-------------+------+-----+---------+-------+
| roll_no   | int         | NO   | PRI | NULL    |       |
| name      | varchar(50) | YES  |     | NULL    |       |
| dob       | date        | YES  |     | NULL    |       |
| address   | text        | YES  |     | NULL    |       |
| phone_no  | varchar(50) | YES  |     | NULL    |       |
| blood_grp | varchar(10) | YES  |     | NULL    |       |
+-----------+-------------+------+-----+---------+-------+
```

6. Drop the column blood_grp from Student table.

>> alter table Student drop column blood_grp;

OUTPUT:

Field	Type	Null	Key	Default	Extra
roll_no	int	NO	PRI	NULL	
name	varchar(50)	YES		NULL	
dob	date	YES		NULL	
address	text	YES		NULL	
phone_no	varchar(50)	YES		NULL	

7. Add a new column Adhaar_no with domain number to the table Student.

>> alter table Student add adhaar_no INT;

OUTPUT:

Field	Type	Null	Key	Default	Extra
roll_no	int	NO	PRI	NULL	
name	varchar(50)	YES		NULL	
dob	date	YES		NULL	
address	text	YES		NULL	
phone_no	varchar(50)	YES		NULL	
adhaar_no	int	YES		NULL	

8. Change the datatype of phone_no from varchar to int

>> alter table Student modify phone_no INT;

OUTPUT:

Field	Type	Null	Key	Default	Extra
roll_no	int	NO	PRI	NULL	
name	varchar(50)	YES		NULL	
dob	date	YES		NULL	
address	text	YES		NULL	
phone_no	int	YES		NULL	
adhaar_no	int	YES		NULL	

9. Drop the tables.

>> drop table Student;
>> drop table Course;

OUTPUT:

```
mysql> DROP TABLE student,course;
Query OK, 0 rows affected (0.31 sec)

mysql> show tables;
Empty set (0.01 sec)
```

10. Delete the database.

>> drop database 25mca4;

OUTPUT:

```
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
| sys                |
+--------------------+
```

B. Consider the database for an organization. Write SQL commands to implement the following:

1. Create a database

>> create database 25mca4;

OUTPUT:

```
+--------------------+
| Database           |
+--------------------+
| 25mca4             |
| information_schema |
| performance_schema |
+--------------------+
```

2. Select the current database

>> USE 25mca4;

OUTPUT:

```
Database changed
```

3. Create the following tables:

a) Employee (emp_no varchar, emp_name varchar, dob date, address text, mobile_no integer, dept_no varchar, salary integer)

>> CREATE TABLE employee(emp_no varchar(20), emp_name varchar(25), dob DATE, address TEXT, mobile_no INT, dept_no VARCHAR(20), salary INT);

b) Department (dept_no varchar, dept_name varchar, location varchar)

>> CREATE TABLE department(dept_no varchar(20), dept_name varchar(25), location VARCHAR(20));

OUTPUT:

```
+------------------+
| Tables_in_25mca4 |
+------------------+
| department       |
| employee         |
+------------------+
```

4. List all tables in the current database.

>> show tables;

OUTPUT:

```
+------------------+
| Tables_in_25mca4 |
+------------------+
| department       |
| employee         |
+------------------+
```

5. Display the structure of the Employee table and Department table.

>> desc Department;
>> desc Employee;

OUTPUT:

```
+-----------+-------------+------+-----+---------+-------+
| Field     | Type        | Null | Key | Default | Extra |
+-----------+-------------+------+-----+---------+-------+
| dept_no   | varchar(20) | YES  |     | NULL    |       |
| dept_name | varchar(25) | YES  |     | NULL    |       |
| location  | varchar(20) | YES  |     | NULL    |       |
+-----------+-------------+------+-----+---------+-------+
```

Field	Type	Null	Key	Default	Extra
emp_no	varchar(20)	YES		NULL	
emp_name	varchar(25)	YES		NULL	
dob	date	YES		NULL	
address	text	YES		NULL	
mobile_no	int	YES		NULL	
dept_no	varchar(20)	YES		NULL	
salary	int	YES		NULL	

6. Add a new column ‘Designation’ to the table Employee.

>> alter table Employee add designation VARCHAR(25);

OUTPUT:

Field	Type	Null	Key	Default	Extra
emp_no	varchar(20)	YES		NULL	
emp_name	varchar(25)	YES		NULL	
dob	date	YES		NULL	
address	text	YES		NULL	
mobile_no	int	YES		NULL	
dept_no	varchar(20)	YES		NULL	
salary	int	YES		NULL	
designation	varchar(25)	YES		NULL	

7. Drop the column ‘location’ from Department table.

>> alter table Department drop column location;

OUTPUT:

Field	Type	Null	Key	Default	Extra
dept_no	varchar(20)	YES		NULL	
dept_name	varchar(25)	YES		NULL	

Date :29/01/2026

Experiment No.02

Aim:Familiarization of SQL Constraints.

SQL constraints are rules applied to columns in a database table to enforce data integrity. They
ensure that the data entered into the table follows specific requirements, such as uniqueness, not
allowing null values, or maintaining referential integrity. Common types of SQL constraints include PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, and CHECK.

1. Create new table Persons with attributes PersonID (integer, PRIMARY KEY), Name (varchar , NOT NULL), Aadhar (Number, NOT NULL, UNIQUE), Age (integer , CHECK>18).

>> create table Persons(person_id int PRIMARY KEY,name varchar(50) NOT NULL,aadhar int NOT NULL UNIQUE,age int CHECK(age>18));

OUTPUT:

```
+-----------+-------------+------+-----+---------+-------+
| Field     | Type        | Null | Key | Default | Extra |
+-----------+-------------+------+-----+---------+-------+
| person_id | int         | NO   | PRI | NULL    |       |
| name      | varchar(50) | NO   |     | NULL    |       |
| aadhar    | int         | NO   | UNI | NULL    |       |
| age       | int         | YES  |     | NULL    |       |
+-----------+-------------+------+-----+---------+-------+
```

2. CREATE TABLE Orders with attributes OrderID (PRIMARY KEY), OrderNumber(NOT NULL) and PersonID(set FOREIGN KEY on attribute PersonID referencing the column PersonId of Person table)

> > create table Orders(order_id int PRIMARY KEY,order_no int NOT NULL,person_id int,FOREIGN KEY(person_id) references Persons(person_id));

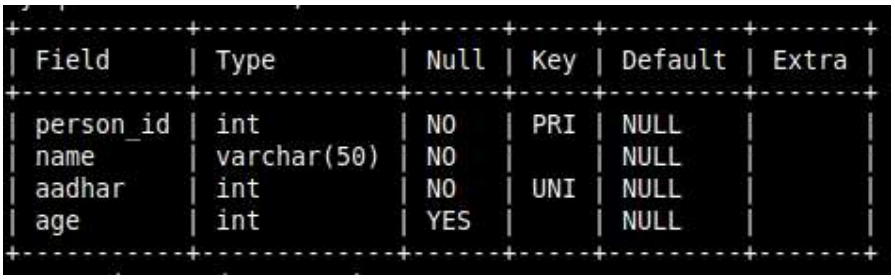
OUTPUT:

```
+-----------+------+------+-----+---------+-------+
| Field     | Type | Null | Key | Default | Extra |
+-----------+------+------+-----+---------+-------+
| order_id  | int  | NO   | PRI | NULL    |       |
| order_no  | int  | NO   |     | NULL    |       |
| person_id | int  | YES  | MUL | NULL    |       |
+-----------+------+------+-----+---------+-------+
```

3. Display the structure of Persons tables.

>> desc Persons;

OUTPUT:

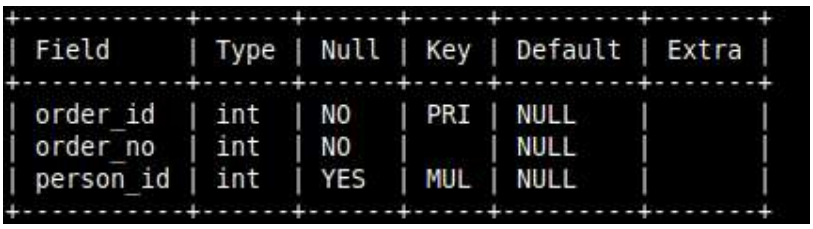


Field	Type	Null	Key	Default	Extra
person_id	int	NO	PRI	NULL	
name	varchar(50)	NO		NULL	
aadhar	int	NO	UNI	NULL	
age	int	YES		NULL	

4. Display the structure of Orders tables.

>> desc Orders;

OUTPUT:

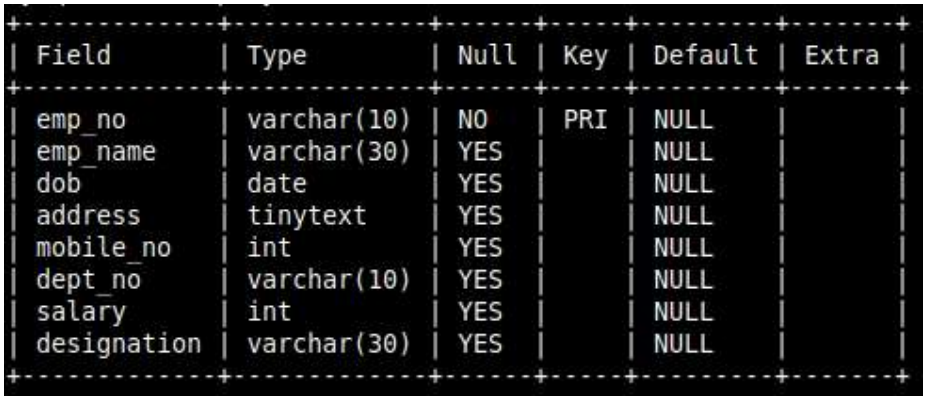


Field	Type	Null	Key	Default	Extra
order_id	int	NO	PRI	NULL	
order_no	int	NO		NULL	
person_id	int	YES	MUL	NULL	

5. Add emp_no as the primary key of the table Employee.

>> alter table Employee add primary key(emp_no);

OUTPUT:



Field	Type	Null	Key	Default	Extra
emp_no	varchar(10)	NO	PRI	NULL	
emp_name	varchar(30)	YES		NULL	
dob	date	YES		NULL	
address	tinytext	YES		NULL	
mobile_no	int	YES		NULL	
dept_no	varchar(10)	YES		NULL	
salary	int	YES		NULL	
designation	varchar(30)	YES		NULL	

6. Add dept_no as the primary key of the table Department.

>> alter table Department add primary key(dept_no);

OUTPUT:

Field	Type	Null	Key	Default	Extra
dept_no	varchar(10)	NO	PRI	NULL	
dept_name	varchar(20)	YES		NULL	

7. Add dept_no in Employee table as the foreign key reference to the table Department with on delete cascade.

>> alter table Employee add foreign key(dept_no) references Department(dept_no) on delete cascade;

OUTPUT:

Field	Type	Null	Key	Default	Extra
emp_no	varchar(10)	NO	PRI	NULL	
emp_name	varchar(30)	YES		NULL	
dob	date	YES		NULL	
address	tinytext	YES		NULL	
mobile_no	int	YES		NULL	
dept_no	varchar(10)	YES	MUL	NULL	
salary	int	YES		NULL	
designation	varchar(30)	YES		NULL	

8. Drop the primary key of the table Orders.

>> alter table Orders drop primary key;

OUTPUT:

Field	Type	Null	Key	Default	Extra
order_id	int	NO		NULL	
order_no	int	NO		NULL	
person_id	int	YES	MUL	NULL	

Date:30/01/2026

Experiment No.3

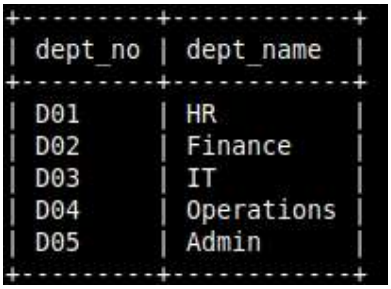
Aim:Familiarization of DML Commands.

To familiarize with Data Manipulation Language (DML) commands in SQL such as INSERT, SELECT, UPDATE, DELETE, and to understand how these commands are used to add, retrieve, modify, and remove data from database tables.

1. Add at least 10 rows into the table Employee and Department.

Department

```
>>INSERT INTO Department VALUES
('D01','HR'),
('D02','Finance'),
('D03','IT'),
('D04','Operations'),
('D05','Admin');
```



dept_no	dept_name
D01	HR
D02	Finance
D03	IT
D04	Operations
D05	Admin

Employee

```
>>INSERT INTO Employee VALUES
('emp1','Arun','1998-05-12','TVM',987654321,'D01',25000,'Manager'),
('emp2','Anita','1999-03-21','KLM',987654322,'D02',18000,'Clerk'),
('emp3','John','1997-07-18','EKM',987654323,'D02',7000,'Assistant'),
('emp4','Asha','1996-11-02','TVM',987654324,'D03',55000,'Manager'),
('emp5','Rahul','1995-01-15','KTM',987654325,'D03',120000,'Developer'),
('emp6','Amit','1998-09-10','ALP',987654326,'D04',30000,'Computer Assistant'),
('emp7','Neha','1999-12-05','PKD',987654327,'D04',45000,'Supervisor'),
('emp8','John','1994-08-25','MLP',987654328,'D05',7000,'Clerk'),
('emp9','Ajay','1993-06-30','TSR',987654329,'D01',90000,'Manager'),
('emp10','Anu','1997-04-14','IDK',987654330,'D02',22000,'Assistant');
```

emp_no	emp_name	dob	address	mobile_no	dept_no	salary	designation
emp1	Arun	1998-05-12	TVM	987654321	D01	25000	Manager
emp10	Anu	1997-04-14	IDK	987654330	D02	22000	Assistant
emp2	Anita	1999-03-21	KLM	987654322	D02	18000	Clerk
emp3	John	1997-07-18	EKM	987654323	D02	7000	Assistant
emp4	Asha	1996-11-02	TVM	987654324	D03	55000	Manager
emp5	Rahul	1995-01-15	KTM	987654325	D03	120000	Developer
emp6	Amit	1998-09-10	ALP	987654326	D04	30000	Computer Assistant
emp7	Neha	1999-12-05	PKD	987654327	D04	45000	Supervisor
emp8	John	1994-08-25	MLP	987654328	D05	7000	Clerk
emp9	Ajay	1993-06-30	TSR	987654329	D01	90000	Manager

2. Display all the records from the above tables.

>> SELECT * FROM Employee;
>> SELECT * FROM Department;

OUTPUT:

emp_no	emp_name	dob	address	mobile_no	dept_no	salary	designation
emp1	Arun	1998-05-12	TVM	987654321	D01	25000	Manager
emp10	Anu	1997-04-14	IDK	987654330	D02	22000	Assistant
emp2	Anita	1999-03-21	KLM	987654322	D02	18000	Clerk
emp3	John	1997-07-18	EKM	987654323	D02	7000	Assistant
emp4	Asha	1996-11-02	TVM	987654324	D03	55000	Manager
emp5	Rahul	1995-01-15	KTM	987654325	D03	120000	Developer
emp6	Amit	1998-09-10	ALP	987654326	D04	30000	Computer Assistant
emp7	Neha	1999-12-05	PKD	987654327	D04	45000	Supervisor
emp8	John	1994-08-25	MLP	987654328	D05	7000	Clerk
emp9	Ajay	1993-06-30	TSR	987654329	D01	90000	Manager

dept_no	dept_name
D01	HR
D02	Finance
D03	IT
D04	Operations
D05	Admin

3. Display the emp_no and name of employees from department no 'D02'

>> SELECT emp_no, emp_name FROM Employee WHERE dept_no = 'D02';

OUTPUT:

emp_no	emp_name
emp10	Anu
emp2	Anita
emp3	John

4. Display emp no, emp_name, designation, deptno and salary of employees in the descending order of salary.

>> SELECT emp_no, emp_name, Designation, dept_no, salary FROM Employee ORDER BY salary DESC;

OUTPUT:

emp_no	emp_name	Designation	dept_no	salary
emp5	Rahul	Developer	D03	120000
emp9	Ajay	Manager	D01	90000
emp4	Asha	Manager	D03	55000
emp7	Neha	Supervisor	D04	45000
emp6	Amit	Computer Assistant	D04	30000
emp1	Arun	Manager	D01	25000
emp10	Anu	Assistant	D02	22000
emp2	Anita	Clerk	D02	18000
emp3	John	Assistant	D02	7000
emp8	John	Clerk	D05	7000

5. Display the emp_no, name of employees whose salary is between 2000 and 5000

>> SELECT emp_no, emp_name FROM Employee WHERE salary BETWEEN 2000 AND 5000;

OUTPUT:

```
Empty set (0.00 sec)
```

6. Display the desigrations without duplicate values

>> SELECT DISTINCT Designation FROM Employee;

OUTPUT:

Designation
Manager
Assistant
Clerk
Developer
Computer Assistant
Supervisor

7. Change the salary of employees to 45000 whose designation is 'Manager'

>> UPDATE Employee SET salary = 45000 WHERE Designation = 'Manager';

OUTPUT:

emp_no	emp_name	dob	address	mobile_no	dept_no	salary	designation
emp1	Arun	1998-05-12	TVM	987654321	D01	45000	Manager
emp10	Anu	1997-04-14	IDK	987654330	D02	22000	Assistant
emp2	Anita	1999-03-21	KLM	987654322	D02	18000	Clerk
emp3	John	1997-07-18	EKM	987654323	D02	7000	Assistant
emp4	Asha	1996-11-02	TVM	987654324	D03	45000	Manager
emp5	Rahul	1995-01-15	KTM	987654325	D03	120000	Developer
emp6	Amit	1998-09-10	ALP	987654326	D04	30000	Computer Assistant
emp7	Neha	1999-12-05	PKD	987654327	D04	45000	Supervisor
emp8	John	1994-08-25	MLP	987654328	D05	7000	Clerk
emp9	Ajay	1993-06-30	TSR	987654329	D01	45000	Manager

8. Change the mobile number of employees named John

>> UPDATE Employee SET mobile_no = 999999999 WHERE emp_name = 'John';

OUTPUT:

emp_no	emp_name	dob	address	mobile_no	dept_no	salary	designation
emp1	Arun	1998-05-12	TVM	987654321	D01	45000	Manager
emp10	Anu	1997-04-14	IDK	987654330	D02	22000	Assistant
emp2	Anita	1999-03-21	KLM	987654322	D02	18000	Clerk
emp3	John	1997-07-18	EKM	999999999	D02	7000	Assistant
emp4	Asha	1996-11-02	TVM	987654324	D03	45000	Manager
emp5	Rahul	1995-01-15	KTM	987654325	D03	120000	Developer
emp6	Amit	1998-09-10	ALP	987654326	D04	30000	Computer Assistant
emp7	Neha	1999-12-05	PKD	987654327	D04	45000	Supervisor
emp8	John	1994-08-25	MLP	999999999	D05	7000	Clerk
emp9	Ajay	1993-06-30	TSR	987654329	D01	45000	Manager

9. Delete all employees whose salary is equal to Rs.7000

>> DELETE FROM Employee WHERE salary = 7000;

OUTPUT:

emp_no	emp_name	dob	address	mobile_no	dept_no	salary	designation
emp1	Arun	1998-05-12	TVM	987654321	D01	45000	Manager
emp10	Anu	1997-04-14	IDK	987654330	D02	22000	Assistant
emp2	Anita	1999-03-21	KLM	987654322	D02	18000	Clerk
emp4	Asha	1996-11-02	TVM	987654324	D03	45000	Manager
emp5	Rahul	1995-01-15	KTM	987654325	D03	120000	Developer
emp6	Amit	1998-09-10	ALP	987654326	D04	30000	Computer Assistant
emp7	Neha	1999-12-05	PKD	987654327	D04	45000	Supervisor
emp9	Ajay	1993-06-30	TSR	987654329	D01	45000	Manager

10. Retrieve the name, mobile number of all employees whose name start with "A".

>> SELECT emp_name, mobile_no FROM Employee WHERE emp_name LIKE 'A%';

OUTPUT:

emp_name	mobile_no
Arun	987654321
Anu	987654330
Anita	987654322
Asha	987654324
Amit	987654326
Ajay	987654329

11. Display the details of the employee whose name has at least three characters and salary greater than 20000.

>> SELECT * FROM Employee WHERE LENGTH(emp_name) >= 3 AND salary > 20000;

OUTPUT:

emp_no	emp_name	dob	address	mobile_no	dept_no	salary	designation
emp1	Arun	1998-05-12	TVM	987654321	D01	45000	Manager
emp10	Anu	1997-04-14	IDK	987654330	D02	22000	Assistant
emp4	Asha	1996-11-02	TVM	987654324	D03	45000	Manager
emp5	Rahul	1995-01-15	KTM	987654325	D03	120000	Developer
emp6	Amit	1998-09-10	ALP	987654326	D04	30000	Computer Assistant
emp7	Neha	1999-12-05	PKD	987654327	D04	45000	Supervisor
emp9	Ajay	1993-06-30	TSR	987654329	D01	45000	Manager

12. Display the details of employees with empid 'empl', 'emp2' and 'emp6"

>> SELECT * FROM Employee WHERE emp_no IN ('emp1','emp2','emp6');

OUTPUT:

emp_no	emp_name	dob	address	mobile_no	dept_no	salary	designation
emp1	Arun	1998-05-12	TVM	987654321	D01	45000	Manager
emp2	Anita	1999-03-21	KLM	987654322	D02	18000	Clerk
emp6	Amit	1998-09-10	ALP	987654326	D04	30000	Computer Assistant

13. Display employee name and employee id of those who have salary between 120000 and 300000.

>> SELECT emp_no, emp_name FROM Employee WHERE salary BETWEEN 120000 AND 300000;

OUTPUT:

emp_no	emp_name
emp5	Rahul

14. Display the details of employees whose designation is 'Manager' or 'Computer Assistant'.

>> SELECT * FROM Employee WHERE Designation IN ('Manager','Computer Assistant');

OUTPUT:

emp_no	emp_name	dob	address	mobile_no	dept_no	salary	designation
emp1	Arun	1998-05-12	TVM	987654321	D01	45000	Manager
emp4	Asha	1996-11-02	TVM	987654324	D03	45000	Manager
emp6	Amit	1998-09-10	ALP	987654326	D04	30000	Computer Assistant
emp9	Ajay	1993-06-30	TSR	987654329	D01	45000	Manager

15. Displays how mary employees work for each department.

>> SELECT dept_no, COUNT(*) AS total_employees FROM Employee GROUP BY dept_no;

OUTPUT:

dept_no	total_employees
D01	2
D02	2
D03	2
D04	2

16. Displays average salary of employees in each department.

>> SELECT dept_no, AVG(salary) AS avg_salary FROM Employee GROUP BY dept_no;

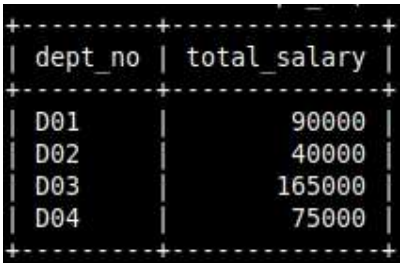
OUTPUT:

dept_no	avg_salary
D01	45000.0000
D02	20000.0000
D03	82500.0000
D04	37500.0000

17. Displays total salary of employees in each department.

>> SELECT dept_no, SUM(salary) AS total_salary FROM Employee GROUP BY dept_no;

OUTPUT:

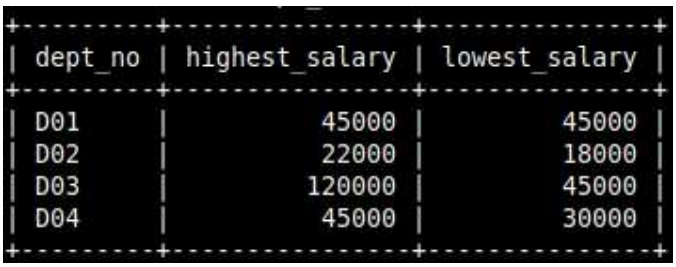


dept_no	total_salary
D01	90000
D02	40000
D03	165000
D04	75000

18. Displays top and lower salary of employees in each department.

>> SELECT dept_no, MAX(salary) AS highest_salary, MIN(salary) AS lowest_salary FROM Employee GROUP BY dept_no;

OUTPUT:

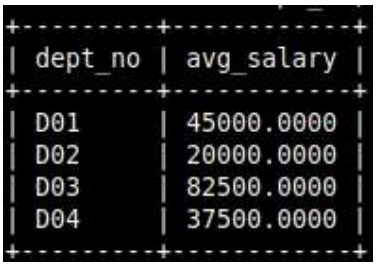


dept_no	highest_salary	lowest_salary
D01	45000	45000
D02	22000	18000
D03	120000	45000
D04	45000	30000

19. Displays average salary of employees in all departments except department with department number 'D05'.

>> SELECT dept_no, AVG(salary) AS avg_salary FROM Employee WHERE dept_no != 'D05' GROUP BY dept_no;

OUTPUT:

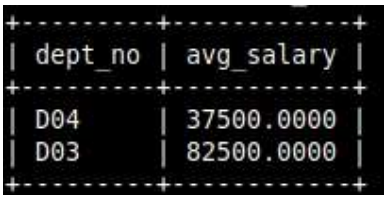


dept_no	avg_salary
D01	45000.0000
D02	20000.0000
D03	82500.0000
D04	37500.0000

20. Displays average salary of employees in all departments except department with department number "D01' and average salary greater than 20000 in the ascending order of average salary.

>>SELECT dept_no, AVG(salary) AS avg_salary FROM Employee WHERE dept_no != 'D01' GROUP BY dept_no HAVING AVG(salary) > 20000 ORDER BY avg_salary ASC;

OUTPUT:



dept_no	avg_salary
D04	37500.0000
D03	82500.0000

LAB CYCLE 2

Date:09/02/2026
Experiment No: 4

AIM: Familiarization of Subquery, Joins, Views and Set Operations.

1. **Subquery**: A subquery is a query nested inside another query, used to retrieve intermediate results that can be utilized by the outer query, often in the SELECT , WHERE , or FROM clauses.

2. **Joins**: Joins are used to combine rows from two or more tables based on a related column, such as an inner join (returning only matching rows) or outer joins (returning all rows from one table and matched rows from the other).

3. **Views**: A view is a virtual table created by a query that defines a specific dataset, allowing users to work with complex queries as if they were regular tables, without altering the underlying data.

4. **Set Operations**: Set operations like UNION , INTERSECT , and EXCEPT are used to combine
results from multiple queries, either by merging rows, finding common rows, or eliminating duplicates between them.

Consider the following Database Schema.
Employee (ID character 5, deptID numeric 2 , Name character 15, Designation character 15, Basic numeric 10,2 , Gender character 1)

```
>> create table Employee(id varchar(5) primary key, deptID int, name varchar(15), designation
varchar(15), basic decimal(10,2), gender varchar(1));
>> INSERT INTO Employee (id, deptID, name, designation, Basic, Gender)
VALUES
('101', 1, 'Ram',     'Typist',  2000.00, 'M'),
('102', 2, 'Arun',    'Analyst', 6000.00, 'M'),
('114', 4, 'Menon',      'Clerk',  1500.00, 'M'),
('115', 4, 'Tim', 'Clerk', 1500.00, 'M'),
('121', 1, 'Ruby', 'Typist', 2010.00, 'F'),
('123', 2, 'Mridula',   'Analyst',   6000.00, 'F'),
('127', 2, 'Kiarn',     'Manager',   4000.00, 'M'),
('156', 3, 'Mary',   'Manager', 4500.00, 'F');
```

id	deptID	name	designation	basic	gender
101	1	Ram	Typist	2000.00	M
102	2	Arun	Analyst	6000.00	M
121	4	Ruby	Typist	2010.00	F
156	4	Mary	Manager	4500.00	F
123	1	Mridula	Analyst	6000.00	F
114	2	Menon	Clerk	1500.00	M
115	2	Tim	Clerk	1500.00	M
127	3	Kiran	Manager	4500.00	M

1.Display the different designations existing in the organisation.

>> select distinct designation from Employee;

OUTPUT:

designation
Typist
Analyst
Manager
Clerk

2. Display the number of different designations existing in the organisation.

>> select designation, count(*) as count from Employee group by designation;

OUTPUT:

designation	count
Typist	2
Analyst	2
Clerk	2
Manager	2

3. Display ID, name, designation,deptID and basic, DA, HRA and net salary of all Employees with suitable headings as DA, HRA and NET_SAL respectively.(DA is 7.5% of basic, and NET_SAL is Basic + DA+ HRA)

>> select id, name, designation, deptID, basic, basic*0.075 as DA, basic*0.1 as HRA, basic*1.175 as NET_SAL from Employee;

OUTPUT:

id	name	designation	deptid	basic	DA	HRA	NET_SAL
101	Ram	Typist	1	2000.00	150.00000	200.000	2350.00000
102	Arun	Analyst	2	6000.00	450.00000	600.000	7050.00000
121	Ruby	Typist	4	2010.00	150.75000	201.000	2361.75000
156	Mary	Manager	4	4500.00	337.50000	450.000	5287.50000
123	Mridula	Analyst	1	6000.00	450.00000	600.000	7050.00000
114	Menon	Clerk	2	1500.00	112.50000	150.000	1762.50000
115	Tim	Clerk	2	1500.00	112.50000	150.000	1762.50000
127	Kiran	Manager	3	4500.00	337.50000	450.000	5287.50000

4. Display the maximum salary given for female Employees.

>> select max(basic) as max_salary from Employee where gender="F";

OUTPUT:

max_salary
6000.00

5. Add a column manager-id into the above table.

>> alter table Employee add column managerid varchar(5);

OUTPUT:

id	deptID	name	designation	basic	gender	managerid
101	1	Ram	Typist	2000.00	M	NULL
102	2	Arun	Analyst	6000.00	M	NULL
121	4	Ruby	Typist	2010.00	F	NULL
156	4	Mary	Manager	4500.00	F	NULL
123	1	Mridula	Analyst	6000.00	F	NULL
114	2	Menon	Clerk	1500.00	M	NULL
115	2	Tim	Clerk	1500.00	M	NULL
127	3	Kiran	Manager	4500.00	M	NULL

6. Update values of manager id of Employees as null for 101, 101 for 102, 121, 156. 102 for 123,114,115.121 for 127.

>> update Employee set managerid='101' where id in ('102','121','156');
>> update Employee set managerid='102' where id in ('123','114','115');
>> update Employee set managerid='121' where id='127';

OUTPUT:

id	deptID	name	designation	basic	gender	managerid
101	1	Ram	Typist	2000.00	M	NULL
102	2	Arun	Analyst	6000.00	M	101
121	4	Ruby	Typist	2010.00	F	101
156	4	Mary	Manager	4500.00	F	101
123	1	Mridula	Analyst	6000.00	F	102
114	2	Menon	Clerk	1500.00	M	102
115	2	Tim	Clerk	1500.00	M	102
127	3	Kiran	Manager	4500.00	M	121

7. Add a column joining date to the above table and update appropriate values for the joining date field.

>> alter table Employee add column joining_date date;
>> update Employee set joining_date='2020-03-15' where id='101';

OUTPUT:

id	deptID	name	designation	basic	gender	managerid	joining_date
101	1	Ram	Typist	2000.00	M	NULL	2020-03-15
102	2	Arun	Analyst	6000.00	M	101	2021-06-20
121	4	Ruby	Typist	2010.00	F	101	2019-09-10
156	4	Mary	Manager	4500.00	F	101	2022-01-05
123	1	Mridula	Analyst	6000.00	F	102	2023-07-18
114	2	Menon	Clerk	1500.00	M	102	2024-08-19
115	2	Tim	Clerk	1500.00	M	102	2022-03-17
127	3	Kiran	Manager	4500.00	M	121	2015-07-25

8. Display the details of Employees according to their seniority.
>> select * from Employee order by joining_date;

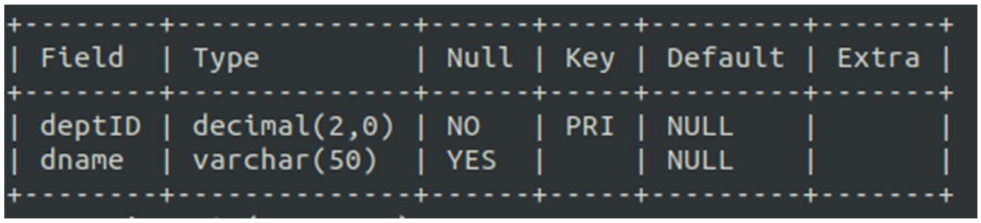
OUTPUT:

id	deptID	name	designation	basic	gender	managerid	joining_date
127	3	Kiran	Manager	4500.00	M	121	2015-07-25
121	4	Ruby	Typist	2010.00	F	101	2019-09-10
101	1	Ram	Typist	2000.00	M	NULL	2020-03-15
102	2	Arun	Analyst	6000.00	M	101	2021-06-20
156	4	Mary	Manager	4500.00	F	101	2022-01-05
115	2	Tim	Clerk	1500.00	M	102	2022-03-17
123	1	Mridula	Analyst	6000.00	F	102	2023-07-18
114	2	Menon	Clerk	1500.00	M	102	2024-08-19

9. Create a new table Department with fields deptID and DNAME. Make deptID as the primary key and make deptID in Employee table to refer to the Department table.

>> create table Department (deptID int(2), dname varchar(50), primary key (deptID));
>> alter table Employee add constraint fk_emp_dept foreign key (deptID) references Department(deptID);

OUTPUT:

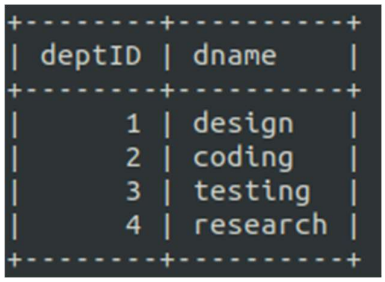


Field	Type	Null	Key	Default	Extra
deptID	decimal(2,0)	NO	PRI	NULL	
dname	varchar(50)	YES		NULL	

10. Insert values into the Department table. Make sure that all the existing values for deptID in emp is inserted into this table. Sample values are DESIGN, CODING,TESTING, RESEARCH.

>> insert into Department values(1,'design'),(2,'coding'),(3,'testing'),(4,'research');

OUTPUT:

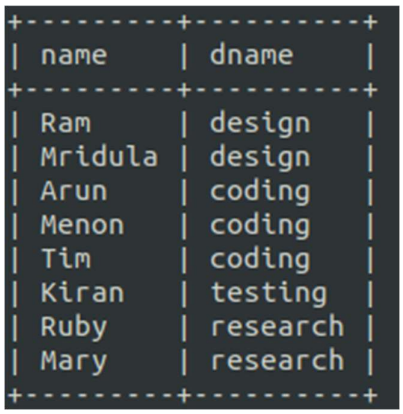


deptID	dname
1	design
2	coding
3	testing
4	research

11. Display the Employee name and Department name.

>> select Employee.name, Department.dname from Employee inner join Department on Employee.deptID = Department.deptID;

OUTPUT:

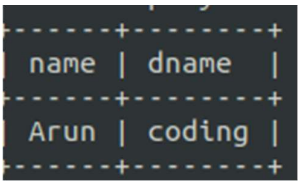


name	dname
Ram	design
Mridula	design
Arun	coding
Menon	coding
Tim	coding
Kiran	testing
Ruby	research
Mary	research

12. Display the Department name of Employee Arun.

>> select Employee.name, Department.dname from Employee join Department on Employee.deptID = Department.deptID where Employee.name="Arun";

OUTPUT:

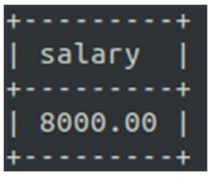


name	dname
Arun	coding

13. Display the salary given by DESIGN Department.

>> select sum(Employee.basic) as salary from Employee join Department on Employee.deptID = Department.deptID where Department.dname="design";

OUTPUT:

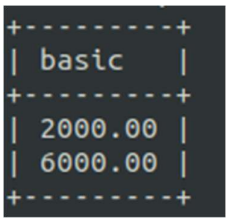


salary
8000.00

14. Display the details of typist working in DESIGN Department.

>> select Employee.basic from Employee join Department on Employee.deptID = Department.deptID where Department.dname="design";

OUTPUT:



basic
2000.00
6000.00

15. Display the salary of Employees working in RESEARCH Department.

>> select Employee.name, Employee.basic from Employee join Department on Employee.deptID = Department.deptID where Department.dname="research";

OUTPUT:

```
+------+---------+
| name | basic   |
+------+---------+
| Ruby | 2010.00 |
| Mary | 4500.00 |
+------+---------+
```

16. List the female Employees working in TESTING Department.

>> select Employee.name from Employee join Department on Employee.deptID = Department.deptID where Department.dname="testing" and gender="F";

OUTPUT:

```
mysql> select Employee.name from Employee join Department on Employee.deptID =
    -> Department.deptID where Department.dname="testing" and gender="F";
Empty set (0.00 sec)
```

17. Display the details of Employees not working in CODING or TESTING Department.

>> select * from Employee join Department on Employee.deptID = Department.deptID where Department.dname not in("Coding","Testing");

OUTPUT:

```
+------+--------+---------+-------------+---------+--------+-----------+--------------+--------+----------+
| id   | deptID | name    | designation | basic   | gender | managerid | joining_date | deptID | dname    |
+------+--------+---------+-------------+---------+--------+-----------+--------------+--------+----------+
| 101  |      1 | Ram     | Typist      | 2000.00 | M      | NULL      | 2020-03-15   |      1 | design   |
| 123  |      1 | Mridula | Analyst     | 6000.00 | F      | 102       | 2023-07-18   |      1 | design   |
| 121  |      4 | Ruby    | Typist      | 2010.00 | F      | 101       | 2019-09-10   |      4 | research |
| 156  |      4 | Mary    | Manager     | 4500.00 | F      | 101       | 2022-01-05   |      4 | research |
+------+--------+---------+-------------+---------+--------+-----------+--------------+--------+----------+
```

18. . Display the names of Department giving maximum salary.

>> select distinct Department.dname from Department join Employee on Department.deptID=Employee.deptID where Employee.basic=(select max(basic) from Employee);

OUTPUT:

```
+--------+
| dname  |
+--------+
| coding |
| design |
+--------+
```

19. Display the names of Departments with minimum number of Employees.

>> select d.dname from Department d join Employee e on d.deptID = e.deptID group by d.dname having count(e.id) = (select min(emp_count) from (select count(id) as emp_count from Employee group by deptID) as subquery);

OUTPUT:

```
+---------+
| dname   |
+---------+
| testing |
+---------+
```

20. Display the second maximum salary.

>> select distinct basic from Employee order by basic desc limit 1 offset 1;

OUTPUT:

```
+---------+
| basic   |
+---------+
| 4500.00 |
+---------+
```

21. Display the second minimum salary.

>> select distinct basic from Employee order by basic asc limit 1 offset 1;

OUTPUT:

```
+---------+
| basic   |
+---------+
| 2000.00 |
+---------+
```

22. Display the names of Employees getting salary greater than the average salary of their Department.

>> select name from employee e1 where basic > (select avg(basic) from employee e2 where e1.deptid = e2.deptid);

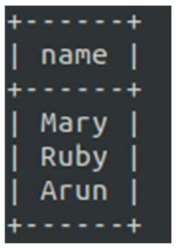
OUTPUT:

```
+---------+
| name    |
+---------+
| Arun    |
| Mary    |
| Mridula |
+---------+
```

23. Display the names of Employees working under the manager Ram.

>> select e.name from Employee e join Employee m on e.managerid = m.id where lower(m.name) = 'ram';

OUTPUT:



```
+------+
| name |
+------+
| Mary |
| Ruby |
| Arun |
+------+
```

LAB CYCLE 3

Date:02/03/2026

Experiment No. 5

AIM: Familiarization of Stored Procedure, Function, Cursor and Triggers

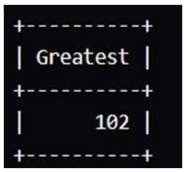
1. **Stored Procedure**: A pre-written set of SQL commands stored in the database, which you can run whenever needed.
2. **Function**: A small program in the database that takes input, processes it, and gives back one result.
3. **Cursor**: A tool in databases used to handle and work with rows of data one at a time.
4. **Trigger**: A rule that automatically runs certain actions in the database when something happens, like adding or updating data

1. Write a stored procedure to read three numbers and find the greatest among them.

SOURCE CODE:

```
USE 25mca4;

DELIMITER //

CREATE PROCEDURE greatest_of_three(IN a INT, IN b INT, IN c INT)
BEGIN
   DECLARE greatest INT;

   IF a >= b AND a >= c THEN
      SET greatest = a;
   ELSEIF b >= a AND b >= c THEN
      SET greatest = b;
   ELSE
      SET greatest = c;
   END IF;

   SELECT greatest AS Greatest_Number;
END //

DELIMITER ;

CALL greatest_of_three(90, 102, 15);
```

OUTPUT:



2. Write a stored procedure to read two numbers and print all the numbers between them.

SOURCE CODE:

```
USE 25mca4;

DELIMITER //
CREATE PROCEDURE print_between(IN a INT, IN b INT)
BEGIN
DECLARE counter INT;
DECLARE RESULT VARCHAR(100);
SET counter =
LEAST(a,b); SET
RESULT = " "; num:
LOOP
SET RESULT = CONCAT(RESULT,counter,' ');
SET counter=counter+1;
IF counter >= GREATEST (a,b) THEN
LEAVE num;
END IF;
END LOOP;
SELECT RESULT AS numbers;
END //
DELIMITER;
CALL print_between(1,10);
```

OUTPUT:

3. Write a stored procedure to read N and find the sum of the series 1+2+3 +... N

SOURCE CODE:

```
USE 25mca4;

DELIMITER //
```

```
CREATE PROCEDURE sum_series(IN n INT)
BEGIN
DECLARE i INT DEFAULT 1;
DECLARE s INT DEFAULT 0;
WHILE i <=n
DO
SET s = s + i;
SET i = i + 1;
END WHILE;
SELECT s AS Sum;
END //
DELIMITER ;
CALL sum_series(10) ;
```

OUTPUT:

4. Write a stored procedure to read a mark and display the grade

SOURCE CODE:

```
USE 25mca4;

DELIMITER //

CREATE PROCEDURE grade_calc(IN marks INT)
BEGIN
   IF marks >= 90 THEN
      SELECT 'A' AS Grade;
   ELSEIF marks >= 75 THEN
      SELECT 'B' AS Grade;
   ELSEIF marks >= 50 THEN
      SELECT 'C' AS Grade;
   ELSE
      SELECT 'Fail' AS Grade;
   END IF;
END //

DELIMITER ;

CALL grade_calc(90);
```

OUTPUT:

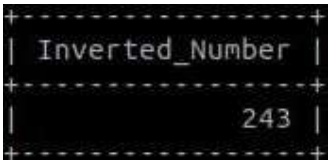
GRADE
A

5. Write a stored procedure to read a number and invert the given number

SOURCE CODE:

```
USE 25mca4;

DELIMITER //

CREATE PROCEDURE invert_number(IN num INT)
BEGIN
   DECLARE rev INT DEFAULT 0;
   DECLARE rem INT;

   WHILE num > 0 DO
      SET rem = num % 10;
      SET rev = rev * 10 + rem;
      SET num = num DIV 10;
   END WHILE;

   SELECT rev AS Reversed_Number;
END //

DELIMITER ;

CALL invert_number(342);
```

OUTPUT:



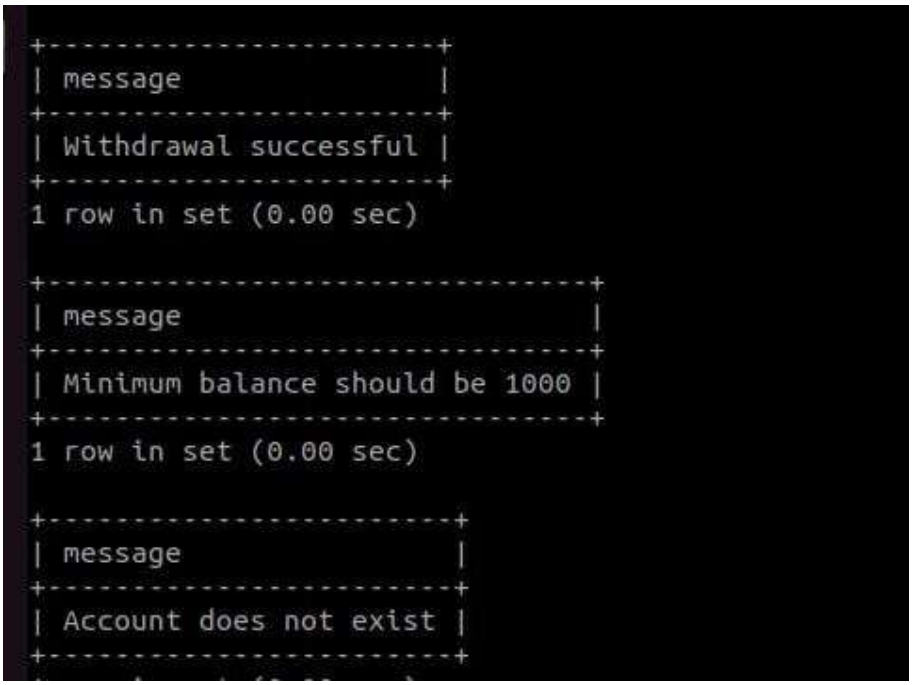
Inverted_Number
243

6. Create a procedure which will receive account_id and amount to withdraw. If the account does not exist, it will display a message. Otherwise, if the account exists, it will allow the withdrawal only if the new balance after the withdrawal is at least 1000.

SOURCE CODE:

```
USE 25mca4;

CREATE TABLE account
(      account_id INT
PRIMARY KEY,
   name
VARCHAR(50),
balance INT
);
INSERT INTO account VALUES (101, 'Ravi', 5000);
INSERT INTO account VALUES (102, 'Anu', 2000);
INSERT INTO account VALUES (103, 'Kiran', 900);
SELECT * FROM account;

DELIMITER //
CREATE PROCEDURE withdraw_amt(
IN acc_id INT,
IN amt INT,
OUT msg VARCHAR(100)
)
BEGIN
DECLARE bal INT;
SELECT balance INTO bal
FROM account
WHERE account_id = acc_id;
IF bal IS NULL THEN
SET msg = 'Account does not exist';
ELSEIF bal - amt < 1000 THEN
SET msg = 'Minimum balance should be 1000';
ELSE
UPDATE account
SET balance = balance - amt

WHERE account_id = acc_id;
SET msg = 'Withdrawal successful';
END IF;
END //
DELIMITER ;
CALL withdraw_amt(101, 1000, @m);
CALL withdraw_amt(109,700,@n);
CALL withdraw_amt(102,2500,@p);

SELECT @m AS message;
SELECT @p AS message;
SELECT @n AS message;
```

OUTPUT:



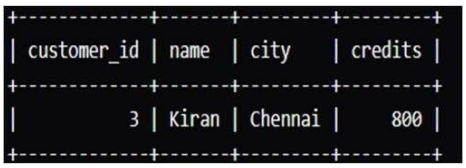
7. Create a ‘Customer’ table with attributes customer id, name, city and credits. Write a stored procedure to display the details of a particular customer from the customer table, where name is passed as a parameter.

SOURCE CODE:

```
USE 25mca4;

CREATE TABLE
Customer(
customer_id INT,
name
VARCHAR(50),
city
VARCHAR(50),
credits INT
);
INSERT INTO Customer VALUES (1, 'Ravi', 'Delhi', 3000);

INSERT INTO Customer VALUES (2, 'Anu', 'Mumbai', 6000);

INSERT INTO Customer VALUES (3, 'Kiran', 'Chennai', 800);

INSERT INTO Customer VALUES (4, 'Meera', 'Bangalore', 4500);

INSERT INTO Customer VALUES (5, 'Arun', 'Hyderabad', 5200);
DELIMITER //
CREATE PROCEDURE customer_details(IN cname VARCHAR(50))
```

```
BEGIN
SELECT *
FROM Customer
WHERE name = cname;
END //
DELIMITER ;
CALL customer_details('Kiran');
```

OUTPUT:



customer_id	name	city	credits
3	Kiran	Chennai	800

8. Create a stored procedure to determine membership of a particular customer based on the following credits: Above 5000 = Membership Platinum 1000 to 5000 = Gold < 1000 = silver [Use IN and OUT Parameters]

SOURCE CODE:

```
USE 25mca4;

DELIMITER //
CREATE PROCEDURE membership(
IN credit INT,
OUT mem VARCHAR(20)
)
BEGIN
IF credit > 5000 THEN
SET mem = 'Platinum';
ELSEIF credit >= 1000 THEN
SET mem = 'Gold';
ELSE
SET mem = 'Silver';
END IF;
END //
DELIMITER ;
CALL membership(3000,@m);
SELECT @m as membership;
```

OUTPUT:

membership
Gold

9. Create a function to accept the Id of an employee and return his salary

SOURCE CODE:

```
USE 25mca4;

CREATE  TABLE  employee
( emp_id INT  PRIMARY KEY,
emp_name VARCHAR(50),
department VARCHAR(50),
salary INT
);

INSERT INTO employee VALUES (101, 'Ravi', 'HR', 25000);
INSERT INTO employee VALUES (102, 'Anu', 'Finance', 30000);
INSERT INTO employee VALUES (103, 'Kiran', 'IT', 35000);
INSERT INTO employee VALUES (104, 'Meera', 'Marketing', 28000);
INSERT INTO employee VALUES (105, 'Arun', 'IT', 40000);
DELIMITER //
CREATE FUNCTION get_salary(eid INT)
RETURNS INT
DETERMINISTIC
BEGIN
DECLARE sal INT;
SELECT salary INTO sal
FROM employee
WHERE emp_id = eid;
RETURN sal;
END //
DELIMITER ;

SELECT get_salary(103);
```

OUTPUT:

```
+------------------+
| get_salary(103) |
+------------------+
|            35000 |
+------------------+
```

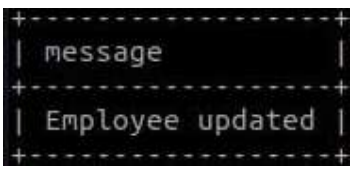
10. Write a function that takes employee name as parameter and returns the number of employees with this name. Use the function to update details of employees with unique names. For other cases, the program (not the function) should display error messages - “No Employee” or “Multiple employees”.

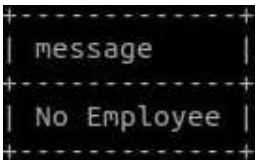
SOURCE CODE:

```
USE 25mca4;
```

```
CREATE TABLE employee2
( emp_id INT  PRIMARY KEY,
emp_name VARCHAR(50),
department VARCHAR(50),
salary INT );

INSERT INTO employee2 VALUES (101, 'Ravi', 'HR', 25000);
INSERT INTO employee2 VALUES (102, 'Anu', 'Finance', 30000);
INSERT INTO employee2 VALUES (103, 'Kiran', 'IT', 35000);
INSERT INTO employee2 VALUES (104, 'Ravi', 'Marketing', 28000);
INSERT INTO employee2 VALUES (105, 'Meera', 'IT', 40000);
DELIMITER //
CREATE FUNCTION emp_count(ename VARCHAR(50))
RETURNS INT
DETERMINISTIC
BEGIN
DECLARE c INT;
SELECT COUNT(*) INTO c
FROM employee
WHERE emp_name = ename;
RETURN c;
END //
DELIMITER ;
DELIMITER //
CREATE PROCEDURE check_employee(
IN ename VARCHAR(50),
OUT msg VARCHAR(50))
BEGIN
DECLARE c INT;
SET c = emp_count(ename);
IF c = 0 THEN
SET msg = 'No Employee';
ELSEIF c > 1 THEN
SET msg = 'Multiple employees';
ELSE
SET msg = 'Employee updated';
END IF;
END //
DELIMITER ;
CALL check_employee('Ravi',@m);
CALL check_employee('Ramu',@n);
SELECT @m as message
```

OUTPUT:





11. Write a stored procedure using cursor to calculate salary of each employee. Consider an Emp_salary table have the following attributes emp_id, emp_name, no_of_working_days,designation and salary

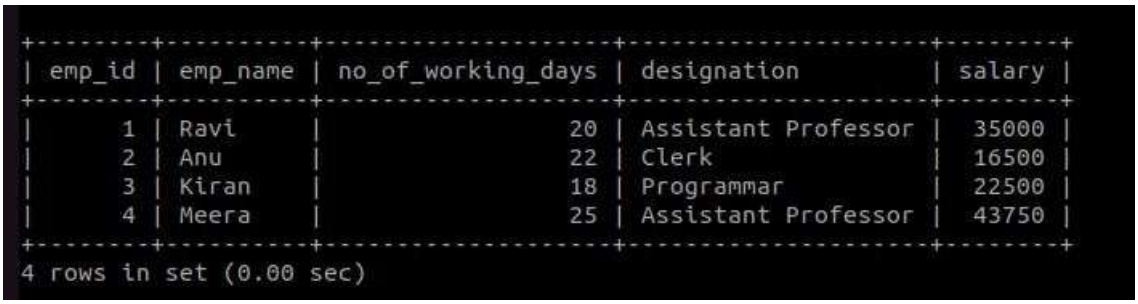
DESIGNATION	DAILY WAGE AMOUNT
Assistant Professor	1750/day
Clerk	750/day
Progarmmar	1250/day

SOURCE CODE:

```
USE 25mca4;

CREATE TABLE
Emp_salary (   emp_id INT,
emp_name VARCHAR(50),
no_of_working_days  INT,
designation VARCHAR(50),
salary INT );
INSERT INTO Emp_salary VALUES (1,'Ravi',20,'Assistant Professor',0);
INSERT INTO Emp_salary VALUES (2,'Anu',22,'Clerk',0);
INSERT INTO Emp_salary VALUES (3,'Kiran',18,'Programmar',0);
INSERT INTO Emp_salary VALUES (4,'Meera',25,'Assistant Professor',0);
DELIMITER //
CREATE PROCEDURE calc_salary()
BEGIN
DECLARE done INT DEFAULT 0;
DECLARE eid INT;
DECLARE days INT;
DECLARE desig VARCHAR(50);
DECLARE sal INT;
DECLARE cur CURSOR FOR
SELECT  emp_id,  no_of_working_days,  designation  FROM
Emp_salary;  DECLARE CONTINUE HANDLER FOR NOT
FOUND SET done = 1; OPEN cur; loop1: LOOP
FETCH cur INTO eid, days, desig;
IF done = 1 THEN
LEAVE loop1;
```

```
END IF;
IF desig = 'Assistant Professor' THEN
SET sal = days * 1750;
ELSEIF desig = 'Clerk' THEN
SET sal = days * 750;
ELSE
SET sal = days * 1250;
END IF;
UPDATE Emp_salary
SET salary = sal
WHERE emp_id = eid;
END LOOP;
CLOSE cur;
END //
DELIMITER ;
CALL calc_salary();
SELECT * FROM Emp_salary;
```

OUTPUT:



```
+--------+----------+--------------------+----------------------+--------+
| emp_id | emp_name | no_of_working_days | designation          | salary |
+--------+----------+--------------------+----------------------+--------+
|      1 | Ravi     |                 20 | Assistant Professor  |  35000 |
|      2 | Anu      |                 22 | Clerk                |  16500 |
|      3 | Kiran    |                 18 | Programmar           |  22500 |
|      4 | Meera    |                 25 | Assistant Professor  |  43750 |
+--------+----------+--------------------+----------------------+--------+
4 rows in set (0.00 sec)
```

12. Write a procedure to calculate the electricity bill of all customers. Electricity board charges the following rates to domestic uses to find the consumption of energy.

a) For first 100 units Rs:2 per unit.

b) 101 to 200 units Rs:2.5 per unit.

c) 201 to 300 units Rs: 3 per unit.

d) Above 300 units Rs: 4 per unit

Consider the table ‘Bill’ with fields customer_id, name, pre_reading, cur_reading , unit, and amount.

SOURCE CODE:

```
USE 25mca4;

CREATE  TABLE
Bill ( customer_id
INT,name
VARCHAR(50),
pre_reading  INT,
```

```
cur_reading
INT, unit INT,
amount FLOAT );
INSERT INTO Bill VALUES (1,'Ravi',1000,1080,0,0);
INSERT INTO Bill VALUES (2,'Anu',2000,2150,0,0);
INSERT INTO Bill VALUES (3,'Kiran',3000,3250,0,0);
INSERT INTO Bill VALUES (4,'Meera',1500,1900,0,0);

DELIMITER //
CREATE PROCEDURE electricity_bill()
BEGIN
UPDATE Bill
SET unit = cur_reading - pre_reading;
UPDATE Bill
SET amount =
CASE
WHEN unit <= 100 THEN unit * 2
WHEN unit <= 200 THEN (100*2) + (unit-100)*2.5
WHEN unit <= 300 THEN (100*2) + (100*2.5) + (unit-200)*3
ELSE (100*2) + (100*2.5) + (100*3) + (unit-300)*4
END;
END //
DELIMITER ;
CALL electricity_bill();
SELECT * FROM Bill;
```

OUTPUT:

```
+-------------+-------+-------------+-------------+------+--------+
| customer_id | name  | pre_reading | cur_reading | unit | amount |
+-------------+-------+-------------+-------------+------+--------+
|           1 | Ravi  |        1000 |        1080 |   80 |    160 |
|           2 | Anu   |        2000 |        2150 |  150 |    325 |
|           3 | Kiran |        3000 |        3250 |  250 |    600 |
|           4 | Meera |        1500 |        1900 |  400 |   1150 |
+-------------+-------+-------------+-------------+------+--------+
4 rows in set (0.00 sec)
```

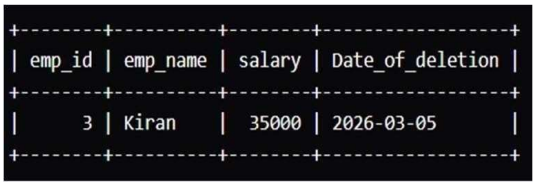
13. Create a trigger on employee table such that whenever a row is deleted, it is moved to history table named ‘Emp_history’ with the same structure as employee table. ‘Emp_history’ will contain an additional column “Date_of_deletion” to store the date on which the row is removed. [After Delete Trigger]

SOURCE CODE:

```
USE 25mca4;

DELIMITER //
CREATE TRIGGER emp_delete
AFTER DELETE ON employee
```

```
FOR EACH ROW
BEGIN
   INSERT INTO Emp_history
   VALUES (OLD.emp_id, OLD.emp_name, OLD.salary, CURDATE());
END //
DELIMITER ;
DELETE FROM employee WHERE emp_id = 3;  SELECT *
FROM Emp_history;
```

OUTPUT:



emp_id	emp_name	salary	Date_of_deletion
3	Kiran	35000	2026-03-05

14. Before insert a new record in emp_details table, create a trigger that check the column value of FIRST_NAME, LAST_NAME, JOB_ID and if there are any space(s) before or after the FIRST_NAME, LAST_NAME, TRIM () function will remove those. The value of the JOB_ID will be converted to upper cases by UPPER () function. [Before Insert Trigger]

SOURCE CODE:

```
USE 25mca4;

DELIMITER //
CREATE TRIGGER emp_trim
BEFORE INSERT ON emp_details
FOR EACH ROW
BEGIN
   SET NEW.first_name = TRIM(NEW.first_name);
   SET NEW.last_name = TRIM(NEW.last_name);
   SET NEW.job_id = UPPER(NEW.job_id);
END //
DELIMITER ;
INSERT INTO emp_details VALUES (1,'  ravi ',' kumar ','clerk');  SELECT *
FROM emp_details;
```

OUTPUT:

```
+--------+------------+-----------+--------+
| emp_id | first_name | last_name | job_id |
+--------+------------+-----------+--------+
|      1 | ravi       | kumar     | CLERK  |
+--------+------------+-----------+--------+
```

15.Consider the following table with sample data. Create a trigger to calculate total marks,percentage and grade of the students, when marks of the subjects are updated. [After Update Trigger]

Stud_id	Name	Sub1	Sub2	Sub3	Sub4	Sub5	Total	Per_Marks	Grade
1	Steven King	0	0	0	0	0	0.0		
2	Neena Kochhar	0	0	0	0	0	0.0		
3	Alexander Hunold	0	0	0	0	0	0.0		
4	Lex De Haan	0	0	0	0	0	0.0		

For this sample calculation, the following conditions are assumed:
Total Marks (will be stored in TOTAL column) : TOTAL = SUB1 + SUB2 + SUB3 + SUB4 + SUB5.
Percentage of Marks (will be stored in PER_MARKS column): PER_MARKS = (TOTAL)/5
Grade (will be stored in GRADE column):
- If PER_MARKS>=90 -> 'EXCELLENT'
- If PER_MARKS>=75 AND PER_MARKS<90 -> 'VERY GOOD'
- If PER_MARKS>=60 AND PER_MARKS<75 -> 'GOOD'
- If PER_MARKS>=40 AND PER_MARKS<60 -> 'AVERAGE'
- If PER_MARKS <40 -> 'NOT PROMOTED'

SOURCE CODE:

```
USE 25mca4;

DELIMITER //
CREATE TRIGGER marks_calc
BEFORE UPDATE ON student
FOR EACH ROW
BEGIN
   SET NEW.total = NEW.sub1 + NEW.sub2 + NEW.sub3 + NEW.sub4 + NEW.sub5
   SET NEW.per_marks = NEW.total / 5;
   IF NEW.per_marks >= 90 THEN
      SET NEW.grade = 'EXCELLENT';
   ELSEIF NEW.per_marks >= 75 THEN
      SET NEW.grade = 'VERY GOOD';
   ELSEIF NEW.per_marks >= 60 THEN
      SET NEW.grade = 'GOOD';
```

```
    ELSEIF NEW.per_marks >= 40 THEN
       SET NEW.grade = 'AVERAGE';
    ELSE
       SET NEW.grade = 'NOT PROMOTED';
    END IF;
END //
DELIMITER ;
UPDATE student
SET sub1=90, sub2=85, sub3=88, sub4=92, sub5=87
WHERE stud_id=1; SELECT *
FROM student;
```

OUTPUT:

stud_id	name	sub1	sub2	sub3	sub4	sub5	total	per_marks	grade
1	Steven King	90	85	88	92	87	442	88.4	VERY GOOD
2	Neena Kochhar	90	95	98	92	97	472	94.4	EXCELLENT
3	Alexander Hunold	60	75	68	62	67	332	66.4	GOOD
4	Lex De Haan	40	55	58	42	57	252	50.4	AVERAGE

LAB CYCLE 4

Date:24/03/2026

Experiment No: 6

AIM : Familiarisation of TCL Commands

Create a Table ‘Order_Details’ with the given data nd perform the following operations

Order_ID	Product_Name	Order_Num	Order_Date
1	Laptop	5544	2020-02-01
2	Mouse	3322	2020-02-11
3	Desktop	2135	2020-01-05
4	Mobile	3432	2020-02-22
5	Anti-Virus	5648	2020-03-10

```
>> CREATE TABLE Order_Details (
   Order_ID INT PRIMARY KEY,
   Product_Name VARCHAR(50),
   Order_Num INT,
   Order_Date DATE
);

>> INSERT INTO Order_Details (Order_ID, Product_Name, Order_Num, Order_Date)
VALUES
(1, 'Laptop', 5544, '2020-02-01'),
(2, 'Mouse', 3322, '2020-02-11'),
(3, 'Desktop', 2135, '2020-01-05'),
(4, 'Mobile', 3432, '2020-02-22'),
(5, 'Anti-Virus', 5648, '2020-03-10');
```

OUTPUT:

```
mysql> select * from Order_Details;
+----------+-------------+-----------+------------+
| Order_ID | Product_Name | Order_Num | Order_Date |
+----------+-------------+-----------+------------+
|        1 | Laptop      |      5544 | 2020-02-01 |
|        2 | Mouse       |      3322 | 2020-02-11 |
|        3 | Desktop     |      2135 | 2020-01-05 |
|        4 | Mobile      |      3432 | 2020-02-22 |
|        5 | Anti-Virus  |      5648 | 2020-03-10 |
+----------+-------------+-----------+------------+
5 rows in set (0.00 sec)

mysql>
```

a) Start a new transaction and insert 2 rows into the table Order_Details.

```
>> START TRANSACTION;
```

> > INSERT INTO Order_Details VALUES (6, 'Keyboard', 7788, '2020-03-15');
>> INSERT INTO Order_Details VALUES (7, 'Monitor', 8899, '2020-03-18');

OUTPUT :

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> INSERT INTO Order_Details VALUES (6, 'Keyboard', 7788, '2020-03-15');
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO Order_Details VALUES (7, 'Monitor', 8899, '2020-03-18');
Query OK, 1 row affected (0.00 sec)

mysql>
```

b) Display the details of Order_Details.

>> SELECT * FROM Order_Details;

OUTPUT:

```
mysql> SELECT * FROM Order_Details;
+----------+--------------+-----------+------------+
| Order_ID | Product_Name | Order_Num | Order_Date |
+----------+--------------+-----------+------------+
|        1 | Laptop       |      5544 | 2020-02-01 |
|        2 | Mouse        |      3322 | 2020-02-11 |
|        3 | Desktop      |      2135 | 2020-01-05 |
|        4 | Mobile       |      3432 | 2020-02-22 |
|        5 | Anti-Virus   |      5648 | 2020-03-10 |
|        6 | Keyboard     |      7788 | 2020-03-15 |
|        7 | Monitor      |      8899 | 2020-03-18 |
+----------+--------------+-----------+------------+
7 rows in set (0.00 sec)

mysql>
```

c) Rollback the current transaction and check the contents of the table Order_Details.

>> ROLLBACK;

>> SELECT * FROM Order_Details;

OUTPUT:

```
mysql> ROLLBACK;
Query OK, 0 rows affected (0.01 sec)

mysql> SELECT * FROM Order_Details;
+----------+--------------+-----------+------------+
| Order_ID | Product_Name | Order_Num | Order_Date |
+----------+--------------+-----------+------------+
|        1 | Laptop       |      5544 | 2020-02-01 |
|        2 | Mouse        |      3322 | 2020-02-11 |
|        3 | Desktop      |      2135 | 2020-01-05 |
|        4 | Mobile       |      3432 | 2020-02-22 |
|        5 | Anti-Virus   |      5648 | 2020-03-10 |
+----------+--------------+-----------+------------+
5 rows in set (0.00 sec)

mysql>
```

d) Start a new transaction and delete 2 rows from the table Order_Details.

sql:-

>> START TRANSACTION;

>> DELETE FROM Order_Details WHERE Order_ID IN (1, 2);

OUTPUT:

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> DELETE FROM Order_Details
    -> WHERE Order_ID IN (1, 2);
Query OK, 2 rows affected (0.00 sec)

mysql>
```

e) Display the details of Order_Details.

>> SELECT * FROM Order_Details;

OUTPUT:

```
mysql> SELECT * FROM Order_Details;
+----------+--------------+-----------+------------+
| Order_ID | Product_Name | Order_Num | Order_Date |
+----------+--------------+-----------+------------+
|        3 | Desktop      |      2135 | 2020-01-05 |
|        4 | Mobile       |      3432 | 2020-02-22 |
|        5 | Anti-Virus   |      5648 | 2020-03-10 |
+----------+--------------+-----------+------------+
3 rows in set (0.00 sec)

mysql>
```

f) Commit the current transaction and check the details of the table Order_Details.

sql:-

>> COMMIT;

>> SELECT * FROM Order_Details;

OUTPUT:

```
mysql> COMMIT;
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT * FROM Order_Details;
+----------+--------------+-----------+------------+
| Order_ID | Product_Name | Order_Num | Order_Date |
+----------+--------------+-----------+------------+
|        3 | Desktop      |      2135 | 2020-01-05 |
|        4 | Mobile       |      3432 | 2020-02-22 |
|        5 | Anti-Virus   |      5648 | 2020-03-10 |
+----------+--------------+-----------+------------+
3 rows in set (0.00 sec)

mysql>
```

g) Disable autocommit and create a savepoint named ‘Check_Updates’.

>> SET autocommit = 0;

>> START TRANSACTION;

>> SAVEPOINT Check_Updates;

OUTPUT:

```
mysql> SET autocommit = 0;
Query OK, 0 rows affected (0.00 sec)

mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> SAVEPOINT Check_Updates;
Query OK, 0 rows affected (0.00 sec)

mysql>
```

h) Update details of the order with Order_Num 5544.

>> UPDATE Order_Details
SET Product_Name = 'Gaming Laptop'
WHERE Order_Num = 5544;

OUTPUT:

```
mysql> UPDATE Order_Details
    -> SET Product_Name = 'Gaming Laptop'
    -> WHERE Order_Num = 5544;
Query OK, 0 rows affected (0.00 sec)
Rows matched: 0  Changed: 0  Warnings: 0

mysql>
```

i) Insert 2 more rows into the table.

>> INSERT INTO Order_Details VALUES (8, 'Printer', 9900, '2020-03-20');
>> INSERT INTO Order_Details VALUES (9, 'Scanner', 9911, '2020-03-21');

OUTPUT :

```
mysql> INSERT INTO Order_Details VALUES (8, 'Printer', 9900, '2020-03-20');
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO Order_Details VALUES (9, 'Scanner', 9911, '2020-03-21');
Query OK, 1 row affected (0.00 sec)

mysql>
```

j) Create a savepoint named ‘Check_delete’

>> SAVEPOINT Check_delete;

OUTPUT:

```
mysql> SAVEPOINT Check_delete;
Query OK, 0 rows affected (0.00 sec)

mysql>
```

k) Delete order details of a product named ‘Laptop’.

>> DELETE FROM Order_Details
WHERE Product_Name = 'Laptop';

OUTPUT:

```
mysql> DELETE FROM Order_Details
    -> WHERE Product_Name = 'Laptop';
Query OK, 0 rows affected (0.00 sec)

mysql>
```

l) Display the current details of the table Order_Details.

>> SELECT * FROM Order_Details;

OUTPUT:

```
mysql> SELECT * FROM Order_Details;
+----------+--------------+-----------+------------+
| Order_ID | Product_Name | Order_Num | Order_Date |
+----------+--------------+-----------+------------+
|        3 | Desktop      |      2135 | 2020-01-05 |
|        4 | Mobile       |      3432 | 2020-02-22 |
|        5 | Anti-Virus   |      5648 | 2020-03-10 |
|        8 | Printer      |      9900 | 2020-03-20 |
|        9 | Scanner      |      9911 | 2020-03-21 |
+----------+--------------+-----------+------------+
5 rows in set (0.00 sec)

mysql>
```

m) Rollback to savepoint ‘Check_delete’ and check the details of the table Order_Details.

>> ROLLBACK TO Check_delete;

>> SELECT * FROM Order_Details;

OUTPUT:

```
mysql> ROLLBACK TO Check_delete;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> SELECT * FROM Order_Details;
+----------+--------------+-----------+------------+
| Order_ID | Product_Name | Order_Num | Order_Date |
+----------+--------------+-----------+------------+
|        3 | Desktop      |      2135 | 2020-01-05 |
|        4 | Mobile       |      3432 | 2020-02-22 |
|        5 | Anti-Virus   |      5648 | 2020-03-10 |
|        8 | Printer      |      9900 | 2020-03-20 |
|        9 | Scanner      |      9911 | 2020-03-21 |
+----------+--------------+-----------+------------+
5 rows in set (0.00 sec)

mysql>
```

n) Rollback to savepoint 'Check_Updates' and check the details of the table Order_Details.

>> ROLLBACK TO Check_Updates;

>> SELECT * FROM Order_Details;

OUTPUT:

```
mysql> ROLLBACK TO Check_Updates;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> SELECT * FROM Order_Details;
+----------+--------------+-----------+------------+
| Order_ID | Product_Name | Order_Num | Order_Date |
+----------+--------------+-----------+------------+
|        3 | Desktop      |      2135 | 2020-01-05 |
|        4 | Mobile       |      3432 | 2020-02-22 |
|        5 | Anti-Virus   |      5648 | 2020-03-10 |
+----------+--------------+-----------+------------+
3 rows in set (0.00 sec)

mysql>
```

o) Commit the changes and enable autocommit.

>> COMMIT;

>> SET autocommit = 1;

OUTPUT:

```
mysql> COMMIT;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> SET autocommit = 1;
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT * FROM Order_Details;
+----------+--------------+-----------+------------+
| Order_ID | Product_Name | Order_Num | Order_Date |
+----------+--------------+-----------+------------+
|        3 | Desktop      |      2135 | 2020-01-05 |
|        4 | Mobile       |      3432 | 2020-02-22 |
|        5 | Anti-Virus   |      5648 | 2020-03-10 |
+----------+--------------+-----------+------------+
3 rows in set (0.00 sec)

mysql>
```

LAB CYCLE 5

Date:06/04/2026

Experiment No: 7

AIM: Familiarisation of MongoDB

Part A: Basic Operations

1. Create a MongoDB database named “Inventory” and switch to it.
>> use Inventory

OUTPUT:

```
test> use inventory
switched to db inventory
inventory>
```

2. Create a collection named ‘Products’ and insert suitable documents with fields and values as follows:
>> db.Products.insertMany([

```
{
 "_id": 1,
 "name": "xPhone",
 "price": 799,
 "releaseDate": ISODate("2011-05-14"),
 "spec": { "ram": 4, "screen": 6.5, "cpu": 2.66 },
 "color": ["white","black"],
 "storage": [64,128,256]
},
{
 "_id": 2,
 "name": "xTablet",
 "price": 899,
 "releaseDate": ISODate("2011-09-01"),
 "spec": { "ram": 16, "screen": 9.5, "cpu": 3.66 },
 "color": ["white","black","purple"],
 "storage": [128,256,512]
},
{
 "_id": 3,
 "name": "SmartTablet",
 "price": 899,
 "releaseDate": ISODate("2015-01-14"),
```

```
  "spec": { "ram": 12, "screen": 9.7, "cpu": 3.66 },
  "color": ["blue"],
  "storage": [16,64,128]
},
{
  "_id": 4,
  "name": "SmartPad",
  "price": 699,
  "releaseDate": ISODate("2020-05-14"),
  "spec": { "ram": 8, "screen": 9.7, "cpu": 1.66 },
  "color": ["white","orange","gold","gray"],
  "storage": [128,256,1024]
},
{
  "_id": 5,
  "name": "SmartPhone",
  "price": 599,
  "releaseDate": ISODate("2022-09-14"),
  "spec": { "ram": 4, "screen": 9.7, "cpu": 1.66 },
  "color": ["white","orange","gold","gray"],
  "storage": [128,256]
}
])
```

OUTPUT:

3. Display all documents in the products collection.

```
>> db.Products.find()
```

OUTPUT:

```
Inventory> db.Products.find()
[
  {
    _id: 1,
    name: 'xPhone',
    price: 799,
    releaseDate: ISODate('2011-05-14T00:00:00.000Z'),
    spec: { ram: 4, screen: 6.5, cpu: 2.66 },
    color: [ 'white', 'black' ],
    storage: [ 64, 128, 256 ]
  },
  {
    _id: 2,
    name: 'xTablet',
    price: 899,
    releaseDate: ISODate('2011-09-01T00:00:00.000Z'),
    spec: { ram: 16, screen: 9.5, cpu: 3.66 },
    color: [ 'white', 'black', 'purple' ],
    storage: [ 128, 256, 512 ]
  },
  {
    _id: 3,
    name: 'SmartTablet',
    price: 899,
    releaseDate: ISODate('2015-01-14T00:00:00.000Z'),
    spec: { ram: 12, screen: 9.7, cpu: 3.66 },
    color: [ 'blue' ],
    storage: [ 16, 64, 128 ]
  },
  {
    _id: 4,
    name: 'SmartPad',
    price: 699,
    releaseDate: ISODate('2020-05-14T00:00:00.000Z'),
    spec: { ram: 8, screen: 9.7, cpu: 1.66 },
    color: [ 'white', 'orange', 'gold', 'gray' ],
    storage: [ 128, 256, 1024 ]
  },
  {
    _id: 5,
    name: 'SmartPhone',
    price: 599,
    releaseDate: ISODate('2022-09-14T00:00:00.000Z'),
    spec: { ram: 4, screen: 9.7, cpu: 1.66 },
    color: [ 'white', 'orange', 'gold', 'gray' ],
    storage: [ 128, 256 ]
  }
]
Inventory>
```

4. Display all details of the product with _id = 2.
>> db.Products.find({ _id: 2 })

OUTPUT:

```
Inventory> db.Products.find({ _id: 2 })
[
  {
    _id: 2,
    name: 'xTablet',
    price: 899,
    releaseDate: ISODate('2011-09-01T00:00:00.000Z'),
    spec: { ram: 16, screen: 9.5, cpu: 3.66 },
    color: [ 'white', 'black', 'purple' ],
    storage: [ 128, 256, 512 ]
  }
]
```

5. Display the first document in the products collection.

>> db.Products.findOne()

OUTPUT:

```
Inventory> db.Products.findOne()
{
  _id: 1,
  name: 'xPhone',
  price: 799,
  releaseDate: ISODate('2011-05-14T00:00:00.000Z'),
  spec: { ram: 4, screen: 6.5, cpu: 2.66 },
  color: [ 'white', 'black' ],
  storage: [ 64, 128, 256 ]
}
```

:-

6. Display only the name and price of the product with _id = 5.

>> db.Products.find({ _id: 5 }, { name: 1, price: 1, _id: 0 })

OUTPUT:

```
Inventory> db.Products.find({ _id: 5 }, { name: 1, price: 1, _id: 0 })
[ { name: 'SmartPhone', price: 599 } ]
```

Part B: Querying Documents

7. Select all documents where the price equals 899.

>> db.Products.find({ price: 899 })

OUTPUT:

```
Inventory> db.Products.find({ price: 899 })
[
  {
    _id: 2,
    name: 'xTablet',
    price: 899,
    releaseDate: ISODate('2011-09-01T00:00:00.000Z'),
    spec: { ram: 16, screen: 9.5, cpu: 3.66 },
    color: [ 'white', 'black', 'purple' ],
    storage: [ 128, 256, 512 ]
  },
  {
    _id: 3,
    name: 'SmartTablet',
    price: 899,
    releaseDate: ISODate('2015-01-14T00:00:00.000Z'),
    spec: { ram: 12, screen: 9.7, cpu: 3.66 },
    color: [ 'blue' ],
    storage: [ 16, 64, 128 ]
  }
]
```

8. Find documents where the ram value inside spec is 4.

>> db.Products.find({ "spec.ram": 4 })

OUTPUT:

9. Find all documents where the color array contains "black".

>> db.Products.find({ color: "black" })

OUTPUT:

10. Select documents where the release date is 2020-05-14.

>> db.Products.find({ releaseDate: ISODate("2020-05-14") })

OUTPUT:

```
Inventory> db.Products.find({ releaseDate: ISODate("2020-05-14") })
[
  {
    _id: 4,
    name: 'SmartPad',
    price: 699,
    releaseDate: ISODate('2020-05-14T00:00:00.000Z'),
    spec: { ram: 8, screen: 9.7, cpu: 1.66 },
    color: [ 'white', 'orange', 'gold', 'gray' ],
    storage: [ 128, 256, 1024 ]
  }
]
```

11. Select documents where the price is less than 799.

>> db.Products.find({ price: { $lt: 799 } })

OUTPUT:

```
Inventory> db.Products.find({ price: { $lt: 799 } })
[
  {
    _id: 4,
    name: 'SmartPad',
    price: 699,
    releaseDate: ISODate('2020-05-14T00:00:00.000Z'),
    spec: { ram: 8, screen: 9.7, cpu: 1.66 },
    color: [ 'white', 'orange', 'gold', 'gray' ],
    storage: [ 128, 256, 1024 ]
  },
  {
    _id: 5,
    name: 'SmartPhone',
    price: 599,
    releaseDate: ISODate('2022-09-14T00:00:00.000Z'),
    spec: { ram: 4, screen: 9.7, cpu: 1.66 },
    color: [ 'white', 'orange', 'gold', 'gray' ],
    storage: [ 128, 256 ]
  }
]
Inventory>
```

12. Select documents where the screen size (inside spec) is less than 7.

>> db.Products.find({ "spec.screen": { $lt: 7 } })

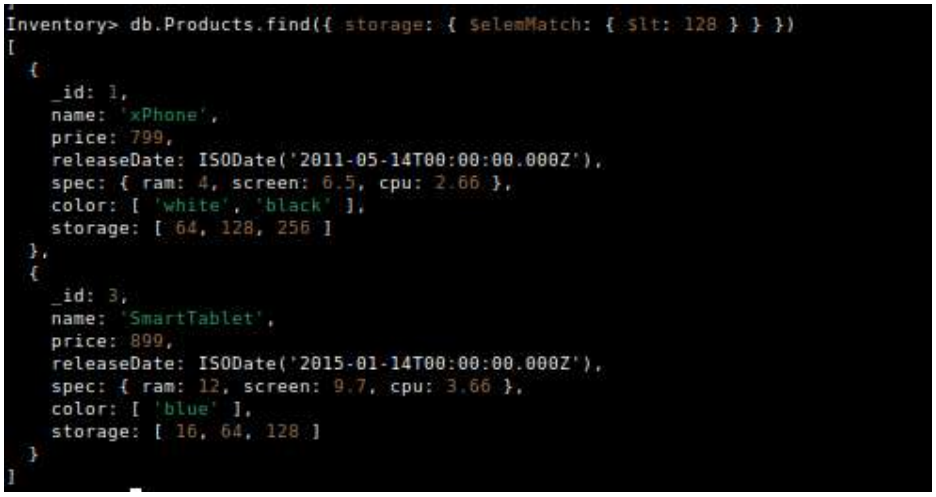
OUTPUT:

```
Inventory> db.Products.find({ 'spec.screen': { $lt: 7 } })
[
  {
    _id: 1,
    name: 'xPhone',
    price: 799,
    releaseDate: ISODate('2011-05-14T00:00:00.000Z'),
    spec: { ram: 4, screen: 6.5, cpu: 2.66 },
    color: [ 'white', 'black' ],
    storage: [ 64, 128, 256 ]
  }
]
Inventory>
```

13. Find documents where the storage array contains at least one value less than 128.

>> db.Products.find({ storage: { $elemMatch: { $lt: 128 } } })

OUTPUT:



Part C: Advanced Operators

14. Display documents where the price is either 699 or 799.

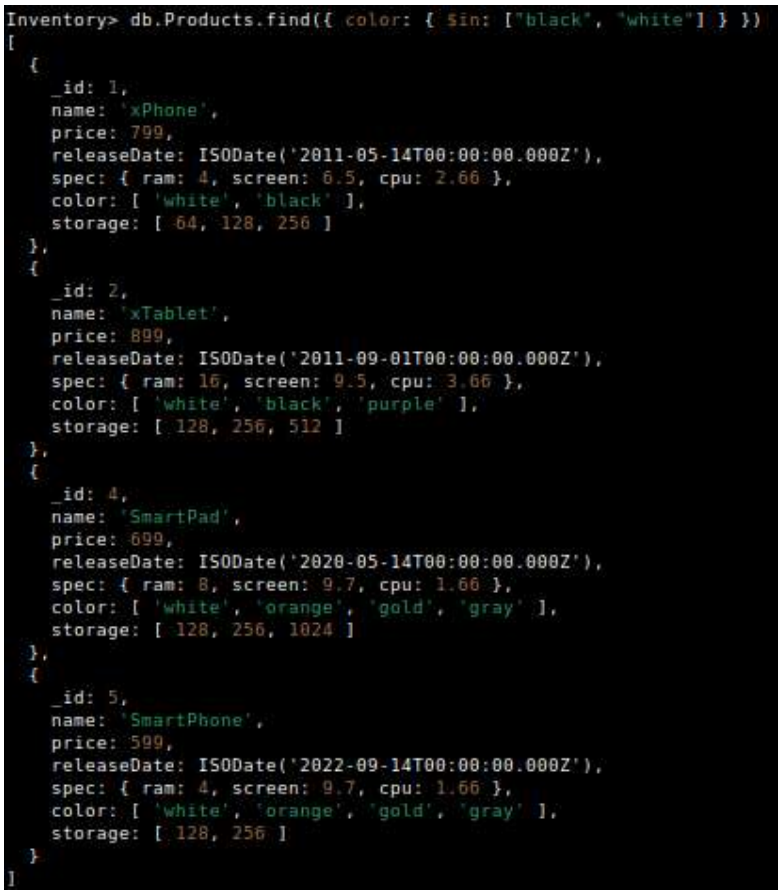
>> db.Products.find({ price: { $in: [699, 799] } })

OUTPUT:

15. Display documents where the color array contains at least one element either "black" or "white".

>> db.Products.find({ color: { $in: ["black", "white"] } })

OUTPUT:



16. Display documents where the price is neither 699 nor 799.

>> db.Products.find({ price: { $nin: [699, 799] } })

OUTPUT:

```
Inventory> db.Products.find({ price: { $nin: [699, 799] } })
[
  {
    _id: 2,
    name: 'xTablet',
    price: 899,
    releaseDate: ISODate('2011-09-01T00:00:00.000Z'),
    spec: { ram: 16, screen: 9.5, cpu: 3.66 },
    color: [ 'white', 'black', 'purple' ],
    storage: [ 128, 256, 512 ]
  },
  {
    _id: 3,
    name: 'SmartTablet',
    price: 899,
    releaseDate: ISODate('2015-01-14T00:00:00.000Z'),
    spec: { ram: 12, screen: 9.7, cpu: 3.66 },
    color: [ 'blue' ],
    storage: [ 16, 64, 128 ]
  },
  {
    _id: 5,
    name: 'SmartPhone',
    price: 599,
    releaseDate: ISODate('2022-09-14T00:00:00.000Z'),
    spec: { ram: 4, screen: 9.7, cpu: 1.66 },
    color: [ 'white', 'orange', 'gold', 'gray' ],
    storage: [ 128, 256 ]
  }
]
Inventory>
```

17. Display documents where the color array does not contain "black" or "white".

>> db.Products.find({ color: { $nin: ["black", "white"] } })

OUTPUT:

```
Inventory> db.Products.find({ color: { $nin: ["black", "white"] } })
[
  {
    _id: 3,
    name: 'SmartTablet',
    price: 899,
    releaseDate: ISODate('2015-01-14T00:00:00.000Z'),
    spec: { ram: 12, screen: 9.7, cpu: 3.66 },
    color: [ 'blue' ],
    storage: [ 16, 64, 128 ]
  }
]
Inventory>
```

18. Display documents where:
• price = 899 AND
• color is either "white" or "black"

```
>> db.Products.find({
  price: 899,
  color: { $in: ["white", "black"] }
})
```

OUTPUT:

```
Inventory> db.Products.find({
|   price: 899,
|   color: { $in: ["white", "black"] }
| })
[
  {
    _id: 2,
    name: 'xTablet',
    price: 899,
    releaseDate: ISODate('2011-09-01T00:00:00.000Z'),
    spec: { ram: 16, screen: 9.5, cpu: 3.66 },
    color: [ 'white', 'black', 'purple' ],
    storage: [ 128, 256, 512 ]
  }
]
Inventory>
```

19. Display documents where:
• price < 699 OR
• price > 799

```
>> db.Products.find({
  $or: [
    { price: { $lt: 699 } },
    { price: { $gt: 799 } }
  ]
})
```

OUTPUT:

```
Inventory> db.Products.find({
|   $or: [
|     { price: { $lt: 699 } },
|     { price: { $gt: 799 } }
|   ]
| })
[
  {
    _id: 2,
    name: 'xTablet',
    price: 899,
    releaseDate: ISODate('2011-09-01T00:00:00.000Z'),
    spec: { ram: 16, screen: 9.5, cpu: 3.66 },
    color: [ 'white', 'black', 'purple' ],
    storage: [ 128, 256, 512 ]
  },
  {
    _id: 3,
    name: 'SmartTablet',
    price: 899,
    releaseDate: ISODate('2015-01-14T00:00:00.000Z'),
    spec: { ram: 12, screen: 9.7, cpu: 3.66 },
    color: [ 'blue' ],
    storage: [ 16, 64, 128 ]
  },
  {
    _id: 5,
    name: 'SmartPhone',
    price: 599,
    releaseDate: ISODate('2022-09-14T00:00:00.000Z'),
    spec: { ram: 4, screen: 9.7, cpu: 1.66 },
    color: [ 'white', 'orange', 'gold', 'gray' ],
    storage: [ 128, 256 ]
  }
]
Inventory>
```

Part D: Sorting

20. Sort the products based on RAM (inside spec) in ascending order. Include the _id, name, and spec fields in the matching documents.

>> db.Products.find({}, { _id: 1, name: 1, spec: 1 })

.sort({ "spec.ram": 1 })

OUTPUT:

```
Inventory> db.Products.find({}, { _id: 1, name: 1, spec: 1 })
| .sort({ "spec.ram": 1 })
[
  { _id: 1, name: 'xPhone', spec: { ram: 4, screen: 6.5, cpu: 2.66 } },
  {
    _id: 5,
    name: 'SmartPhone',
    spec: { ram: 4, screen: 9.7, cpu: 1.66 }
  },
  { _id: 4, name: 'SmartPad', spec: { ram: 8, screen: 9.7, cpu: 1.66 } },
  {
    _id: 3,
    name: 'SmartTablet',
    spec: { ram: 12, screen: 9.7, cpu: 3.66 }
  },
  { _id: 2, name: 'xTablet', spec: { ram: 16, screen: 9.5, cpu: 3.66 } }
]
Inventory>
```

21. Sort the products by release date in descending order.

>> db.Products.find().sort({ releaseDate: -1 })

OUTPUT:

```
Inventory> db.Products.find().sort({ releaseDate: -1 })
[
  {
    _id: 5,
    name: 'SmartPhone',
    price: 599,
    releaseDate: ISODate('2022-09-14T00:00:00.000Z'),
    spec: { ram: 4, screen: 9.7, cpu: 1.66 },
    color: [ 'white', 'orange', 'gold', 'gray' ],
    storage: [ 128, 256 ]
  },
  {
    _id: 4,
    name: 'SmartPad',
    price: 699,
    releaseDate: ISODate('2020-05-14T00:00:00.000Z'),
    spec: { ram: 8, screen: 9.7, cpu: 1.66 },
    color: [ 'white', 'orange', 'gold', 'gray' ],
    storage: [ 128, 256, 1024 ]
  },
  {
    _id: 3,
    name: 'SmartTablet',
    price: 899,
    releaseDate: ISODate('2015-01-14T00:00:00.000Z'),
    spec: { ram: 12, screen: 9.7, cpu: 3.66 },
    color: [ 'blue' ],
    storage: [ 16, 64, 128 ]
  },
  {
    _id: 2,
    name: 'xTablet',
    price: 899,
    releaseDate: ISODate('2011-09-01T00:00:00.000Z'),
    spec: { ram: 16, screen: 9.5, cpu: 3.66 },
    color: [ 'white', 'black', 'purple' ],
    storage: [ 128, 256, 512 ]
  },
  {
    _id: 1,
    name: 'xPhone',
    price: 799,
    releaseDate: ISODate('2011-05-14T00:00:00.000Z'),
    spec: { ram: 4, screen: 6.5, cpu: 2.66 },
    color: [ 'white', 'black' ],
    storage: [ 64, 128, 256 ]
  }
]
Inventory>
```

22. Sort the products by price and name in ascending order (select only documents where price exists and include the _id, name, and price fields in the matching documents).

```
>> db.Products.find(

  { price: { $exists: true } },
  { _id: 1, name: 1, price: 1 }
).sort({ price: 1, name: 1 })
```

OUTPUT:

```
Inventory> db.Products.find(
|   { price: { $exists: true } },
|   { _id: 1, name: 1, price: 1 }
| ).sort({ price: 1, name: 1 })
[
  { _id: 5, name: 'SmartPhone', price: 599 },
  { _id: 4, name: 'SmartPad', price: 699 },
  { _id: 1, name: 'xPhone', price: 799 },
  { _id: 3, name: 'SmartTablet', price: 899 },
  { _id: 2, name: 'xTablet', price: 899 }
]
Inventory>
```

Part E: Application-Based Query

23. Find the most expensive product in the collection. Include the _id, name, and price fields in the returned documents.

```
>> db.Products.find(

  {},
  { _id: 1, name: 1, price: 1 }
).sort({ price: -1 }).limit(1)
```

OUTPUT:

```
Inventory> db.Products.find(
|   {},
|   { _id: 1, name: 1, price: 1 }
| ).sort({ price: -1 }).limit(1)
[ { _id: 2, name: 'xTablet', price: 899 } ]
Inventory>
```